



Práctica 3

Refactoring CuOOra

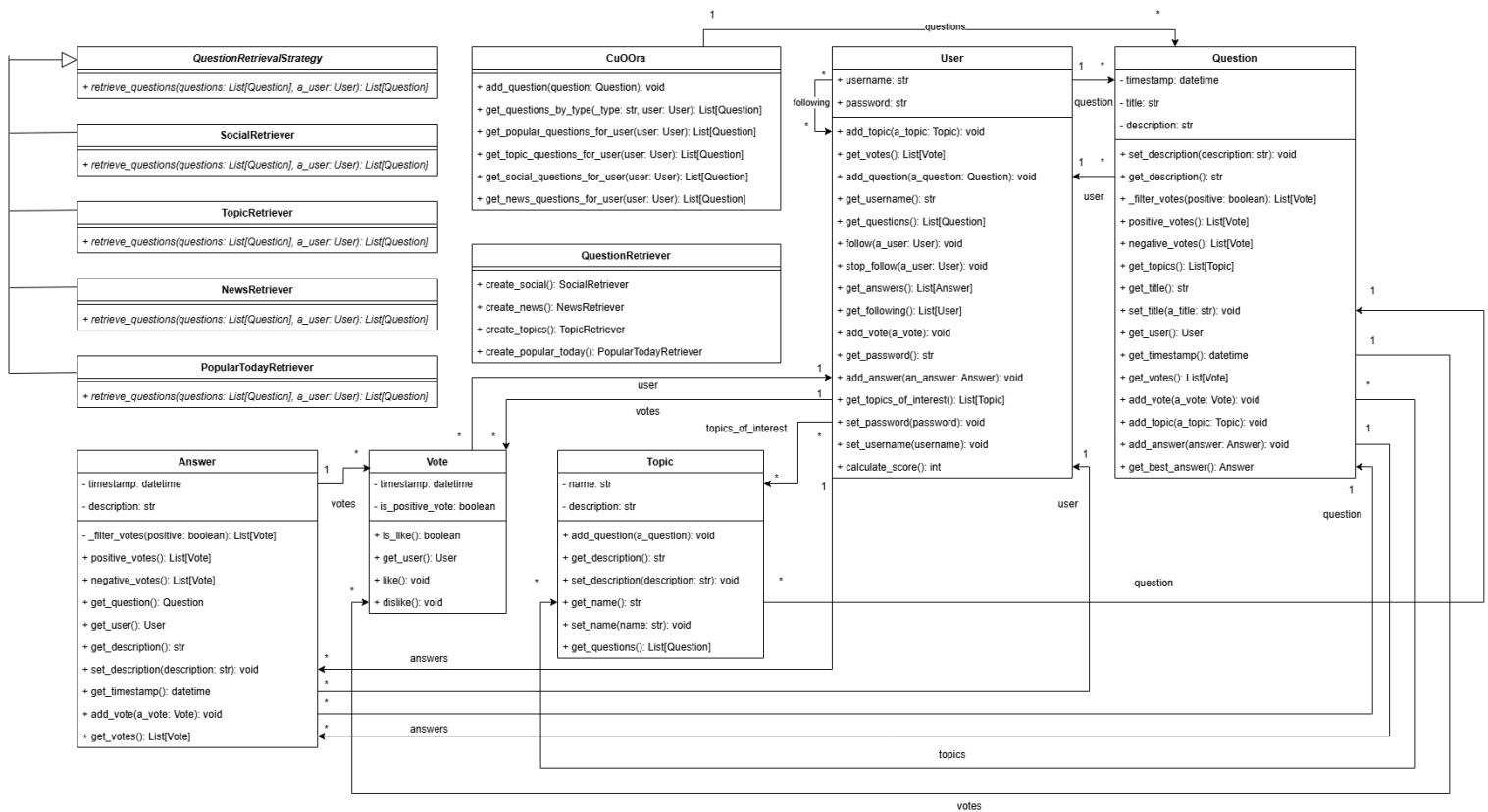
Modelado y diseño de sistemas software

Curso académico 2024/2025

[Enlace GitHub](#)

José María García Gutiérrez, José Miguel Ojeda Gutiérrez.

Diagrama de clases



Este es nuestro diagrama de clases, debido a su tamaño, a continuación te envío el link para poder visualizarlo en la web de draw.io: [Diagrama de clase](#)

Refactoring

1) Clase Answer:

Código Inicial:

```
class Answer(object):
    def __init__(self, question, user, description):
        self.votes = []
        self.timestamp = datetime.now()
        self.user = user
        self.description = description
        self.question = question
        self.question.add_answer(self)

    def positive_votes(self):
        r = []
        for vote in self.votes:
            if vote.is_like():
                r.append(vote)
        return r

    def negative_votes(self):
        r = []
        for vote in self.votes:
            if not vote.is_like():
                r.append(vote)
        return r
```

Code Smells:

1. Nombres poco descriptivos: La variable r no es clara en positive_votes y negative_votes.
2. Código duplicado: Ambas funciones recorren self.votes usando una logica parecida.

Refactoring Aplicado: Extract Method

Creamos un método _filter_votes(is_positive: bool) para que el código sea más claro, y se evite código duplicado

Código final:

```
def _filter_votes(self, positive: bool):
    """Filtra los votos en función de si son positivos o negativos."""
    return [vote for vote in self.votes if vote.is_like() == positive]

def positive_votes(self):
    return self._filter_votes(True)

def negative_votes(self):
    return self._filter_votes(False)
```

2) Clase User

Código Inicial:

```
class User(object):
    def __init__(self, username, password):
        self.username = username
        self.password = password
        self.questions = []
        self.answers = []
        self.topics_of_interest = []
        self.following = []
        self.votes = []

    def calculate_score(self):
        score = 0
        for question in self.questions:
            pv = len(question.positive_votes())
            nv = len(question.negative_votes())
            if pv > nv:
                score += 10

        for answer in self.answers:
            pv = len(answer.positive_votes())
            nv = len(answer.negative_votes())
            if pv > nv:
                score += 20

        return score
```

Code Smells:

1. Código repetitivo: Se calculan las puntuaciones de preguntas y respuestas con la misma lógica, creando código duplicado.
2. Variables innecesarias (pv, nv).

Refactoring Aplicado: Replace Loop with sum()

Código final:

```
def calculate_score(self):
    question_score = sum(10 for q in self.questions if len(q.positive_votes()) > len(q.negative_votes()))
    answer_score = sum(20 for a in self.answers if len(a.positive_votes()) > len(a.negative_votes()))
    return question_score + answer_score
```

Además, se elimina la clase

```
def questions_of_interest(self):
    return self.question_retriever.retrieve_questions(self)
```

Por un Feature Envy

3) Clase QuestionRetriever

Código Inicial:

```
class QuestionRetriever:

    def retrieve_questions(self, questions, a_user):
        q_ret = []
        if self.option == "social":
            following_col = []
            for follow in a_user.following:
                following_col.extend(follow.questions)
            temp = sorted(following_col, key=lambda q: len(q.positive_votes()))
            q_ret = temp[-min(100, len(temp)):]

        elif self.option == "topics":
            topics_col = []
            for topic in a_user.get_topics_of_interest():
                topics_col.extend(topic.questions)
            temp = sorted(topics_col, key=lambda q: len(q.positive_votes()))
            q_ret = temp[-min(100, len(temp)):]

        elif self.option == "news":
            news_col = []
            for question in questions:
                if question.timestamp.date() == datetime.today().date():
                    news_col.append(question)
            temp = sorted(news_col, key=lambda q: len(q.positive_votes()))
            q_ret = temp[-min(100, len(temp)):]

        elif self.option == "popularToday":
            today_questions = [q for q in questions if q.timestamp.date() == datetime.today().date()]
            if today_questions:
                average_votes = sum(len(q.positive_votes()) for q in today_questions) / len(today_questions)
                popular_questions = []
                for q in today_questions:
                    if len(q.positive_votes()) > average_votes:
                        popular_questions.append(q)
                temp = sorted(popular_questions, key=lambda q: len(q.positive_votes()))
                q_ret = temp[-min(100, len(temp)):]

        return [q for q in q_ret if q.user != a_user]
```

Code Smells:

Refactor Retriever:

Bad Smell: la clase QuestionRetriever figuraba originalmente con un largo método que disponía de lógica condicional estructurada de forma similar a un switch statement con el fin de recuperar listas de preguntas según un parámetro tipo. Esta estructura multi-ramal dificulta el mantenimiento del programa.

Solución: Para esto, hemos creado una clase abstracta QuestionRetrievalStrategy con un solo método abstracto "retrieve_questions", que deberá ser redefinido en cada una de las subclases (SocialRetriever, TopicRetriever, NewsRetriever y PopularTodayRetriever). Dicho método contendrá la lógica específica relacionada a cada tipo (social, topics, news, popular_today). Así, la clase QuestionRetriever

actuará como una factoría (se hace uso del patrón Factory Method, en cierta medida), devolviendo la estrategia indicada (patrón Strategy) con sus métodos estilo create(cls).

Conclusión: este refactor se complementa en esencia con el otro refactor de la clase CuOOra, el cuál brindaba un método get_questions_by_type para hacer uso de la recuperación de preguntas de forma organizada. En conjunto obtenemos una lógica compartimentada, un mejor mantenimiento, más ágil testing y más facilidad a la hora de extender la aplicación.

Código Final:

```
class QuestionRetrievalStrategy(ABC):
    @abstractmethod
    def retrieve_questions(self, questions, a_user):
        pass

class SocialRetriever(QuestionRetrievalStrategy):
    def retrieve_questions(self, questions, a_user):
        following_questions = []
        for follow in a_user.following:
            following_questions.extend(follow.questions)
        sorted_q = sorted(following_questions, key=lambda q: len(q.positive_votes()))
        q_ret = sorted_q[-min(100, len(sorted_q)):]
        return [q for q in q_ret if q.user != a_user]

class TopicRetriever(QuestionRetrievalStrategy):
    def retrieve_questions(self, questions, a_user):
        topics_questions = []
        for topic in a_user.get_topics_of_interest():
            topics_questions.extend(topic.questions)
        sorted_q = sorted(topics_questions, key=lambda q: len(q.positive_votes()))
        q_ret = sorted_q[-min(100, len(sorted_q)):]
        return [q for q in q_ret if q.user != a_user]

class NewsRetriever(QuestionRetrievalStrategy):
    def retrieve_questions(self, questions, a_user):
        news_questions = [q for q in questions if q.timestamp.date() == datetime.today().date()]
        sorted_q = sorted(news_questions, key=lambda q: len(q.positive_votes()))
        q_ret = sorted_q[-min(100, len(sorted_q)):]
        return [q for q in q_ret if q.user != a_user]

class PopularTodayRetriever(QuestionRetrievalStrategy):
    def retrieve_questions(self, questions, a_user):
        today_questions = [q for q in questions if q.timestamp.date() == datetime.today().date()]
        if today_questions:
            average_votes = sum(len(q.positive_votes()) for q in today_questions) / len(today_questions)
            popular_questions = [q for q in today_questions if len(q.positive_votes()) > average_votes]
            sorted_q = sorted(popular_questions, key=lambda q: len(q.positive_votes()))
            q_ret = sorted_q[-min(100, len(sorted_q)):]
        else:
            q_ret = []
        return [q for q in q_ret if q.user != a_user]
```

4) Clase Question

Código Inicial:

```
class Question:
    def __init__(self, user, title, description, topics=[]):
        self.timestamp = datetime.now()
        self.title = title
        self.description = description
        self.answers = []
        self.votes = []
        self.user = user
        self.user.add_question(self)
        self.topics = []
        for topic in topics:
            self.add_topic(topic)

    def positive_votes(self):
        r = []
        for vote in self.votes:
            if vote.is_like():
                r.append(vote)
        return r

    def negative_votes(self):
        r = []
        for vote in self.votes:
            if not vote.is_like():
                r.append(vote)
        return r
```

Code Smells:

1. Nombres poco descriptivos: La variable r no es clara en positive_votes y negative_votes.
2. Código duplicado: Ambas funciones recorren self.votes usando una logica parecida.

Refactoring Aplicado: Extract Method

Creamos un método _filter_votes(is_positive: bool) al igual que en la clase "Answer" para que el código sea más claro, y se evite código duplicado

Código final:

```
class Question:
    def __init__(self, user, title, description, topics=[]):
        self.timestamp = datetime.now()
        self.title = title
        self.description = description
        self.answers = []
        self.votes = []
        self.user = user
        self.user.add_question(self)
        self.topics = []
        for topic in topics:
            self.add_topic(topic)

    def _filter_votes(self, positive: bool):
        """Filtra los votos en función de si son positivos o negativos."""
        return [vote for vote in self.votes if vote.is_like() == positive]

    def positive_votes(self):
        return self._filter_votes(True)
```

5) Clase CuOOra

Código Inicial:

```
class CuOOra:
    def __init__(self):
        self.questions = []

    def add_question(self, a_question):
        self.questions.append(a_question)

    def get_social_questions_for_user(self, user):
        social_retriever = QuestionRetriever.create_social()
        return social_retriever.retrieve_questions(self.questions, user)

    def get_topic_questions_for_user(self, user):
        topic_retriever = QuestionRetriever.create_topics()
        return topic_retriever.retrieve_questions(self.questions, user)

    def get_news_questions_for_user(self, user):
        news_retriever = QuestionRetriever.create_news()
        return news_retriever.retrieve_questions(self.questions, user)

    def get_popular_questions_for_user(self, user):
        popular_retriever = QuestionRetriever.create_popular_today()
        return popular_retriever.retrieve_questions(self.questions, user)
```

Code Smells:

Hay una duplicación de código, se repiten los mismos patrones en los métodos `get_social_questions_for_user`, `get_topic_questions_for_user`, `get_news_questions_for_user` y `get_popular_questions_for_user`. Cada uno crea una instancia de un `QuestionRetriever` específico y luego llama al método `retrieve_questions`.

Refactorización aplicada:

1. Introducir un diccionario de métodos (Replace Conditional with Polymorphism)

Se agrupan las diferentes instancias de `QuestionRetriever` en un diccionario para evitar la repetición de código y hacer la selección de manera más limpia y mantenible.

2. Extract Method

Creamos el método `get_questions_by_type`, que recibe el tipo de pregunta y el usuario. Así, cada método específico solo necesita llamar a esta nueva función.

Código Final:

```
class Cu00ra:
    def __init__(self):
        self.questions = []

    def add_question(self, question):
        self.questions.append(question)

    def get_questions_by_type(self, type_, user):
        retriever_methods = {
            "social": QuestionRetriever.create_social,
            "topic": QuestionRetriever.create_topics,
            "news": QuestionRetriever.create_news,
            "popular": QuestionRetriever.create_popular_today,
        }
        if type_ not in retriever_methods:
            raise ValueError("Tipo de pregunta no válido")
        retriever = retriever_methods[type_]()
        return retriever.retrieve_questions(self.questions, user)

    def get_social_questions_for_user(self, user):
        return self.get_questions_by_type("social", user)

    def get_topic_questions_for_user(self, user):
        return self.get_questions_by_type("topic", user)

    def get_news_questions_for_user(self, user):
        return self.get_questions_by_type("news", user)

    def get_popular_questions_for_user(self, user):
        return self.get_questions_by_type("popular", user)
```